

The data model

Three schemas in one Postgres database, named so that anyone opening it understands the product before it is explained. Bronze holds what the files said. Silver holds the rules. Gold holds the numbers. This guide covers every table and view, and how each one is built from the one below it.

One sentence version. Bronze is the only thing stored. Silver and Gold are SQL views, so a new upload or a changed threshold recomputes everything on the next query — there is nothing to refresh and nothing to go stale.

1 · The stack at a glance

BRONZE

stored · immutable

source_file · reading · site · file_overlap

One row per value read out of a spreadsheet cell, with the file, the row and the column it came from. Nothing is ever updated or deleted here.

▼ views, recomputed on every query

SILVER

views · the rules

generation_daily · site_day_expected · data_quality · quality_flag

eligibility · site_month · fleet_month

Every rule is one readable CASE expression. This is the layer shown to a verifier.

▼ views, recomputed on every query

GOLD

views · the numbers

factor_by_month · carbon · vcu · price_by_month · revenue

fleet_summary · fleet_table · lineage

Emission reductions, whole VCUs with carry-forward, indicative revenue — and the lineage that walks any of them back to a cell.

The schema names are the demo. Someone looking at the database should see bronze, silver and gold and understand the product before anyone explains it.

2 · Bronze — what the files said

Four tables. These are the only stored data in the system.

bronze.source_file — one row per uploaded file

Column	Holds
<code>file_id</code>	Surrogate key, referenced by every reading
<code>filename, bytes</code>	As uploaded
<code>sha256</code>	Unique. This is what makes a repeat upload detectable
<code>grain</code>	plant · inverter · unknown
<code>sheet_name, header_row</code>	Where in the workbook the parser found the data
<code>period_start, period_end</code>	The dates the file actually covers
<code>row_count, values_kept, values_blank</code>	The parse summary shown on the upload screen
<code>storage_path</code>	Where the original bytes were written
<code>parse_notes</code>	Anything recognised but unused, or unreadable — as JSON

bronze.reading — long format, one row per cell

Exactly what the parser emits. This shape is what makes lineage possible: every value knows the file, the row and the column it came from.

Column	Holds
<code>reading_id</code>	Surrogate key
<code>file_id</code>	Which upload this came from
<code>source_row, source_col</code>	The cell, as a human counts them — row 3, column BK
<code>plant_name, device_sn</code>	Who generated it
<code>reading_date</code>	The day it refers to
<code>metric, metric_label</code>	Canonical key, plus the raw header text as written in the file
<code>value, unit</code>	The number and its unit

Immutable, deliberately. Nothing ever updates a row here. Re-uploading a period inserts a new file and new readings; which one wins is decided in Silver, and the loser stays visible as superseded.

bronze.site — the fleet master

Derived from the plant information columns and upserted on every load. A load never overwrites a known value with a blank one, so a device report that omits capacity cannot wipe the capacity a plant list supplied.

`plant_name` (key) · `installed_kwp` · `plant_type` · `grid_connection_date` · `address` · `plant_status` · `first_seen_file_id` · `last_seen_file_id`

bronze.file_overlap

Records that a new upload covered dates an earlier one already had. The overlap is recorded rather than resolved by deletion; Silver decides which reading wins.

3 · Silver — the rules

Views only. Every rule lives in a readable SQL statement, because this is the layer that gets shown to a verifier. They build on each other in the order below.

3.1 · silver.file_site_day

Each file's own account of a site-day. Inverter-grain files are summed across devices; plant-grain files pass the plant row straight through.

```
bronze.reading + bronze.source_file → one row per (file, site, day)
```

3.2 · silver.generation_daily

One row per site per day. Where two files cover the same site-day, the most recently uploaded one wins; the loser is kept and exposed as silver.superseded_site_day. This is how supersession works without ever deleting anything.

```
DISTINCT ON (plant_name, reading_date) ORDER BY uploaded_at DESC
```

3.3 · silver.site_day_expected

A row per site per day across the whole loaded window, whether or not data exists. Without this, gaps are invisible — a missing day simply would not appear. This is the view that makes absence visible.

```
bronze.site CROSS JOIN generate_series(window_start, window_end)
```

3.4 · silver.data_quality

One boolean per rule, each its own CASE expression that can be read aloud.

Flag	True when
is_missing	No row exists for that site on that day
is_zero	Generation is exactly zero
is_negative	Generation is below zero
is_implausible	Specific yield exceeds the configured threshold
is_before_grid_connection	The day precedes the site's grid connection date
is_duplicate	Two files disagree about this site-day by more than 0.5%
is_site_inactive	The plant status is not a running one

Why is_duplicate means disagreement, not presence. Two files covering one site-day is normal — the later one wins and the earlier is kept. It is only a quality problem when the two sources disagree about the number. An earlier version flagged every day of a site that appeared in two exports, excluding a whole quarter for no reason.

3.5 · silver.quality_flag

Collapses those booleans into one flag per site-day — ok, suspect or missing — plus the reason in words. A missing day stays missing: it is never interpolated and never quietly becomes a zero.

3.6 · silver.eligibility — the bridge

The view that connects energy to credits. Per site per day, generation is split into eligible and excluded, and every excluded kWh carries a reason.

```
generation_kwh    =    eligible_kwh    +    excluded_kwh

exclusion_reason ∈ { before_grid_connection, site_inactive,
                    data_gap, quality_suspect }
```

Rules are applied in that order and the first match decides. This is what the river on the Overview screen draws, and the tests assert that the two sides balance exactly.

3.7 · silver.site_month and silver.fleet_month

Monthly aggregates that carry the worst child quality, never the average. One missing day makes the month "missing" — an average would let a good month hide a bad week.

4 · Gold — the numbers

4.1 · gold.factor_by_month

Resolves which published emission factor applies to each month. Only the row marked active can drive a number; the other published margins are recorded as reference. If the month falls outside the factor's published validity, this view returns none — and no reduction is claimed.

4.2 · gold.carbon

Eligible MWh times the factor, joined on date so the factor's version travels with the result instead of being baked into it. Project emissions and leakage are explicit zero columns so the formula reads complete.

```
eligible_MWh × emission_factor - project_emissions - leakage = tCO2e
```

4.3 · gold.vcu

A verified carbon unit is one whole tonne, so a month issues whole tonnes and the fraction is carried forward rather than rounded away. The ledger is the running total: each month issues the difference between this month's floor and last month's.

Column	Holds
<code>net_reduction_tco2e</code>	This month's reduction
<code>cumulative_tco2e</code>	Running total for the site
<code>carry_in_tco2e</code>	The fraction brought forward from last month
<code>vcu_issued</code>	Whole units issuable this month
<code>carry_forward_tco2e</code>	The fraction carried on to the next

4.4 · gold.price_by_month and gold.revenue

The VCU price in force at each month's end, and units times price. With no price on file, no revenue figure is produced at all.

4.5 · gold.fleet_summary and gold.fleet_table

The headline row on the Overview screen, and one row per site for the Fleet screen — coverage, specific yield, quality score, VCUs.

4.6 · gold.lineage — the one that matters

Takes any gold figure and returns the chain back to the evidence. It was built at the same time as the calculations, not afterwards.

```
gold.vcu → silver.eligibility → bronze.reading → bronze.source_file  
month → site-days → cells → filename + sha256
```

5 · Why it is views all the way up

- **A new upload changes every number automatically.** There is no pipeline to re-run and no job to schedule.
- **A changed threshold changes every number automatically.** Thresholds live in `silver.parameter`, which the views read, so the number on screen is the number the SQL used.

- **A new emission factor applies from its own date.** Historical months keep the factor that was valid then, because the join is on date.
- **Nothing can go stale,** because nothing downstream of Bronze is stored.

The cost of that choice, and how it was paid. Recomputing on every read is only viable if the views are cheap. They were not at first: a correlated subquery in `data_quality` re-aggregated all of Bronze once per site-day, and the summary endpoint took eight seconds on 676 readings. Resolving that per-day work once, and resolving the factor and price once per month rather than per row, brought it to under half a second. Any new view should be measured the same way.

6 · Dependency map

Read upwards: each view is built from the ones above it.

View	Built from	Layer
<code>file_site_day</code>	<code>bronze.reading</code> + <code>source_file</code>	Silver
<code>generation_daily</code>	<code>file_site_day</code>	Silver
<code>site_day_expected</code>	<code>bronze.site</code> + <code>loaded_window</code>	Silver
<code>data_quality</code>	<code>site_day_expected</code> + <code>generation_daily</code> + <code>site_day_sources</code> + rules	Silver
<code>quality_flag</code>	<code>data_quality</code> + rules	Silver
<code>eligibility</code>	<code>quality_flag</code>	Silver
<code>site_month / fleet_month</code>	<code>eligibility</code>	Silver
<code>factor_by_month</code>	<code>site_month</code> + <code>gold.emission_factor</code>	Gold
<code>carbon</code>	<code>site_month</code> + <code>factor_by_month</code>	Gold
<code>vcu</code>	<code>carbon</code>	Gold
<code>revenue</code>	<code>vcu</code> + <code>price_by_month</code>	Gold
<code>fleet_summary / fleet_table</code>	<code>site_month</code> + <code>vcu</code> + <code>revenue</code>	Gold
<code>lineage</code>	<code>vcu</code> + <code>eligibility</code> + <code>bronze.reading</code> + <code>source_file</code>	Gold

7 · Where to read the source

File	Contains
<code>app/sql/000_reset_views.sql</code>	Drops the view graph so a migration can change a column's shape
<code>app/sql/001_bronze.sql</code>	The four Bronze tables and <code>silver.parameter</code>

File	Contains
<code>app/sql/002_silver.sql</code>	Every rule, as SQL
<code>app/sql/003_gold.sql</code>	Reference tables, the credit maths, and lineage
<code>app/ingest/parser.py</code>	The one parser, imported by both the upload endpoint and the CLI
<code>app/ingest/loader.py</code>	Duplicate detection, overlap flagging, site upsert
<code>tests/test_pipeline.py</code>	The numbers, asserted against a real database

The SQL files are the specification. They are written to be read aloud, and they are the same statements that produced every number on the screen.